Cahier de décisions et de compréhension

Phase 1 — SSVI, acquisition de données, calibration. Document de travail : à remplir à la main.

Ce cahier n'est pas un cours. C'est un **formulaire**. Chaque page pose une question précise et laisse la place d'y répondre. Le cours (`docs/cours_complet.pdf`) explique ; ce cahier sert à vérifier que **tu** peux réexpliquer, et à figer les décisions au fur et à mesure.

Version du 10 novembre 2026. État mesuré de la base au moment de la rédaction : voir §2.3.

Table des matières

- 0. Mode d'emploi
- 1. La structure générale
- 2. Le processus, étape par étape
- 3. Les structures de données
- 4. La base de données
- 5. Les décisions ouvertes
- 6. Fiches fonction — à réécrire à la main
- 7. Journal de session
- Annexe A — Runbook de l'incident du 10 novembre 2026
- Annexe B — Commandes utiles
- Annexe C — Où est quoi
- Annexe D — Glossaire minimal
- Annexe E — Checklist de fin de phase 1

0. Mode d'emploi

Trois types de fiches, trois usages :

Fiche	Ce que tu fais	Quand
DÉCISION (partie 5)	tu écris les options, le critère, la mesure, le choix	une fois, puis tu la cites dans les commits
STRUCTURE (partie 3)	tu redessines la structure de mémoire, puis tu compares	après chaque lecture de code
FONCTION (partie 6)	tu réécris le corps de la fonction en pseudo-code	avant de la modifier

Deux règles qui font la différence :

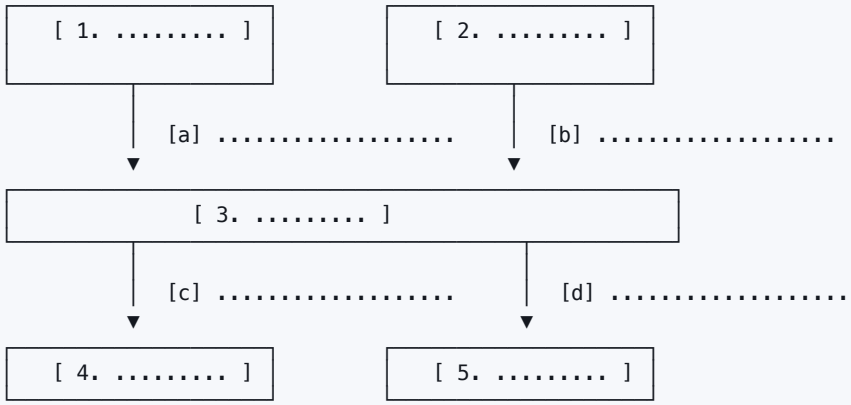
1. **Une décision a un numéro et une date.** « D3 : convention de prix = inverse, 12/11, mesuré sur BTC-25DEC26 » vaut mieux que « on a décidé de passer en inverse ». Un commit qui applique D3 cite D3 ; dans six mois, tu sais pourquoi la ligne existe.
2. **Ce qui n'est pas mesuré n'est pas décidé.** Une case « décidé » sans chiffre dans la colonne « mesure » est une hypothèse déguisée. On l'écrit « hypothèse », pas « décision ».

---

1. La structure générale

1.1 Le schéma à compléter

Le schéma ci-dessous est **volontairement muet** : les cases sont vides. À remplir de mémoire, puis à comparer avec le cours (§1). L'objectif est de pouvoir le redessiner sur une feuille blanche.




```
[1] ..........................................................................
[2] ..........................................................................
[3] ..........................................................................
[4] ..........................................................................
[5] ..........................................................................
[a] ..........................................................................
[b] ..........................................................................
[c] ..........................................................................
[d] ..........................................................................
```

1.2 Les quatre questions qui structurent tout

Réponds sans regarder le cours. Une réponse fausse ici est un trou de compréhension, pas une faute.

Q1. Quelles sont les deux moitiés du projet, et qu'est-ce qui les relie aujourd'hui ?

```
..........................................................................
..........................................................................
..........................................................................
```

Q2. Pourquoi la surface SSVI est-elle indépendante de la convention de règlement (linéaire / inverse) ?

```
..........................................................................
..........................................................................
```

Q3. Où sont les quatre points de variation entre un monde « actions » et un monde « crypto » ?

```
[1] ..........................................................................
[2] ..........................................................................
[3] ..........................................................................
[4] ..........................................................................
```

Q4. Qu'est-ce qui, dans le pipeline actuel, n'est PAS au bon endroit pour un fit défendable ?

```
..........................................................................
..........................................................................
```

> Réponses attendues : cours §1.2 (deux moitiés), §6.1 (SSVI en variance totale), §5.6 (les quatre points), §5.2 (k construit sur le sous-jacent, pas sur le forward).

1.3 Le même schéma, mais vu par les données

```
bronze              silver                    gold                moteur
──────              ──────                    ────                ──────
fichiers zstd  →    instrument (SCD2)     →   v_spot          →   [ ? ]
(rejouables)        option_mark (faits)       v_surface
                    perp_tick                 v_atm_terme
                    vol_index                 v_perp_funding
```

```
Ce qui traverse la dernière flèche, et sous quelle forme :
.....................................................................................
.....................................................................................
```

---

2. Le processus, étape par étape

2.1 Les douze étapes, de la trame WebSocket au paramètre en base

Coche au fur et à mesure que tu sais expliquer l'étape **sans le code**. La colonne de droite est à remplir : c'est là que se trouve ta compréhension.

#	Étape	Fichier	Ce qui peut casser	Je sais l'expliquer
1	souscription WebSocket, 4 canaux	`python/collect/deribit_ws.py`	canal muet, reconnexion en boucle	☐
2	écriture bronze (zstd, rotation horaire)	idem	disque plein, fichier `.part` orphelin	☐
3	lecture du bronze, fenêtre glissante	`python/jobs/load_marks.py`	fenêtre trop courte → trou	☐
4	parsing des trames, échantillonnage 5 min	idem	horodatage fournisseur vs local	☐
5	résolution / création d'instrument (SCD2)	idem + `silver.instrument`	deux versions courantes	☐
6	insertion des faits (idempotente)	`silver.option_mark`	doublon au rejeu, unités mélangées	☐
7	lecture gold (`v_surface`)	`sql/008_couches.sql`	`DISTINCT ON` lent, k faux	☐
8	export du snapshot	*(absent)*	—	☐
9	reconstruction du forward F(T)	*(absent)*	—	☐
10	inversion IV / lecture des IV	*(absent côté crypto)*	unités % vs fraction	☐
11	calibration SSVI	`src/calibration/SSVI.cpp`	borne d'arbitrage, θ figé	☐
12	écriture des paramètres	*(absent : tables supprimées en 004)*	—	☐

```
Les trois étapes manquantes, et pourquoi elles manquent :
.....................................................................................
.....................................................................................
```

2.2 Le déroulé d'une journée

```
00:00 UTC  .....................................................................
08:00 UTC  .....................................................................
toutes les 5 min  ..............................................................
toutes les 10 min ..............................................................
02:30      .....................................................................
dimanche 04:00 .................................................................
```

> À vérifier dans `crontab -l` sur le mini : guard (5 min), load-marks (5 min), backup (02:30), backup bronze (horaire à :05), alerte (10 min), vérification de restauration (dimanche 04:00).

2.3 État mesuré (10 novembre 2026)

Grandeur	Valeur	Comment elle a été obtenue
taille de la base	5,8 Go	`pg_database_size('volsurface')`
`silver.instrument`	4 123 lignes	`count(*)`
`silver.option_mark`	1 395 783 lignes	`count(*)`
`silver.perp_tick`	12 017 lignes	`count(*)`
`silver.vol_index`	12 013 lignes	`count(*)`
`gold.v_surface`	4 120 lignes, 2 sous-jacents, 33 échéances	`count(*)`, `count(distinct ...)`
dernière donnée silver	**2026-11-03 07:30 UTC**	`max(observed_at)`
volume Colima	59 Go, 21 Go utilisés	`df -h /var/lib/docker`

```
Ce que m'apprend l'écart entre la dernière donnée (3 nov.) et aujourd'hui (10 nov.) :
.....................................................................................
.....................................................................................
```

> Réponse attendue : le disque du VM Docker était plein — neuf bases `load_test_*` laissées par les tests d'intégration, 34 Go à elles seules — et PostgreSQL refusait de démarrer. Le collecteur, lui, n'a jamais arrêté : il écrit dans `data/`, hors du VM. Donc **bronze intact, silver en retard**. Leçon : un collecteur « healthy » ne dit rien de la chaîne. Le seul indicateur qui compte est `max(observed_at)`.

2.4 Rejouer une journée à l'identique

```
Ce qui garantit qu'un rejeu du bronze redonne les mêmes lignes :
.....................................................................................
.....................................................................................
La commande, et la fenêtre à choisir :
.....................................................................................
```

---

3. Les structures de données

3.1 L'inventaire

Quinze structures, trois familles. Coche celles que tu peux dessiner de mémoire (champs nommés, types, invariants) sans ouvrir le header.

Famille « entrée » — ce qui vient du marché

Structure	Rôle en une phrase	Fichier	☐
`OptionQuote`	une cotation brute, telle que lue dans le CSV	`include/data/MarketData.hpp`	☐
`MarketData`	le contenu d'un dossier de marché (spot, courbes, quotes)	idem	☐
`RateCurve`	taux zéro par maturité, interpolation linéaire	`include/data/RateCurve.hpp`	☐
`DivCurve`	dividendes discrets, modèle affine partiel	`include/data/DivCurve.hpp`	☐
`SurfaceSnapshot`	ce que le moteur consomme : k, T, matrice d'IV	`include/model/SurfaceSnapshot.hpp`	☐

Famille « modèle » — ce qui produit une valeur

Structure	Rôle	Fichier	☐
`SSVIParams`	rho, eta, gamma, nu	`include/calibration/SSVI.hpp`	☐
`BSParams`	S, K, T, r, q, sigma	`include/pricing/blackscholes.hpp`	☐
`IVResult` / `IVError`	sigma + itérations, ou la raison de l'échec	`include/calibration/ImpliedVol.hpp`	☐
`SSVIError`	trois modes d'échec de la calibration	`include/calibration/SSVI.hpp`	☐

Famille « sortie » — ce qu'on mesure et ce qu'on stocke

Structure	Rôle	Fichier	☐
`FitDiagnostics`	RMSE, ATM, ailes, eta/eta_max, min g	`include/calibration/FitDiagnostics.hpp`	☐
`SliceFitSummary`	le même, pour une maturité	idem	☐
`Grid` / `Tridiag`	grille en S et système tridiagonal	`include/pde/Grid.hpp` , `Tridiag.hpp`	☐
`PDEParams` / `PDEResult`	entrées et sortie du pricer	`include/pde/PDEPricer.hpp`	☐
`Simplex` / `Vertex` / `NMParams`	état de Nelder-Mead (inutilisé)	`include/opt/NelderMead.hpp`	☐

3.2 Fiche structure — à remplir

Prends `SurfaceSnapshot`. Écris ses champs de mémoire, puis ouvre le header.

```
SurfaceSnapshot
─────────────────────────────────────────────────────────────────────────
champ 1 : ..............................  type : ..............  rôle : ..............
champ 2 : ..............................  type : ..............  rôle : ..............
champ 3 : ..............................  type : ..............  rôle : ..............
champ 4 : ..............................  type : ..............  rôle : ..............
─────────────────────────────────────────────────────────────────────────
invariant 1 : ..................................................................
invariant 2 : ..................................................................
qui la remplit : ...............................................................
qui la lit : ...................................................................
ce qui casse si elle est mal formée : ..........................................
```

3.3 Les trois questions d'invariant

I1. Que signifie exactement `iv` vide, `weight` vide, et pourquoi `weight` existe ?

```
.....................................................................................
.....................................................................................
```

I2. Pourquoi la grille en k doit-elle être la même pour toutes les maturités ?

```
.....................................................................................
.....................................................................................
```

I3. Quelle information manque aujourd'hui dans `SurfaceSnapshot` pour refaire un fit à l'identique ?

```
.....................................................................................
.....................................................................................
```

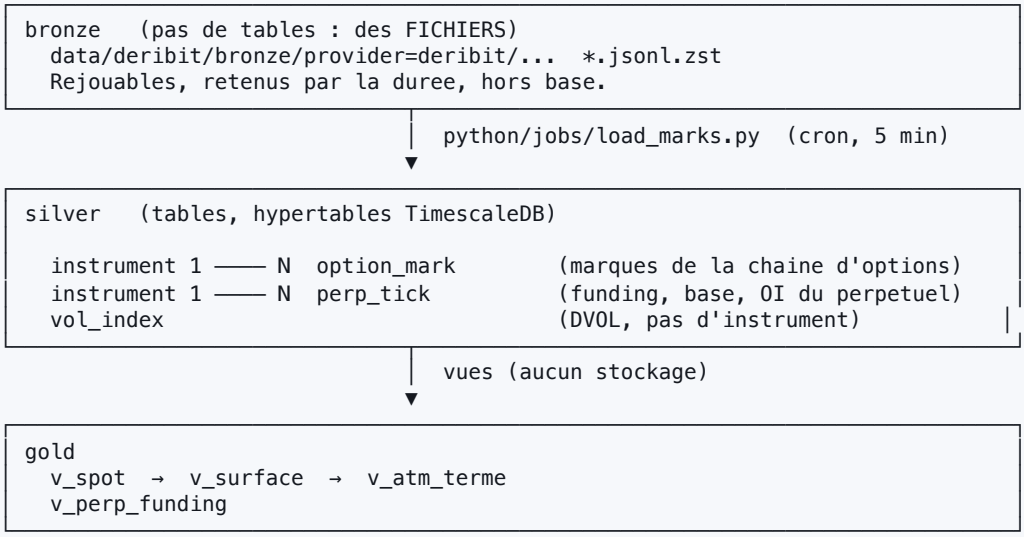
> Attendu pour I3 : l'identité du snapshot (d'où viennent ces chiffres), les bid/ask ou les poids (le fit n'est pas reproductible sans eux), et le forward utilisé pour construire k.

---

4. La base de données

C'est la partie que tu dois pouvoir expliquer de bout en bout, table par table. Elle est plus petite qu'elle n'en a l'air : **4 tables et 4 vues**, plus un registre de migrations.

4.1 La forme générale




```
public.schema_migrations  : le registre du migrateur (sha256 de chaque fichier applique)
```

```
Pourquoi le bronze n'est pas une table de la base :
.....................................................................................
.....................................................................................
```

> Attendu : ~18 Mo/h de messages bruts, contre quelques Mo/jour d'états échantillonnés ; la base ne doit contenir que ce qu'on interroge. Le fichier est rejouable à l'identique, la table ne l'est pas.

4.2 Table `silver.instrument` — la dimension

Grain : une ligne = une version de la définition d'un instrument.

Colonne	Type	Unité / sens	Écrite par	Piège
`instrument_sk`	bigserial	clé de substitution	la base	jamais exposée à l'extérieur
`provider`	text	`deribit`	chargeur	défaut figé
`underlying`	text	`BTC`, `ETH`	chargeur	en majuscules
`instrument_name`	text	`BTC-27MAR26-80000-P`	chargeur	l'identité métier
`kind`	text	option / perpetual / future	chargeur	CHECK
`expiry`	date	échéance (options)	chargeur	**NULL interdit pour un future** (voir §4.9)
`strike`	numeric(18,6)	**USD**	chargeur	jamais en coin
`option_type`	text	`C` / `P`	chargeur	pas `call` / `put`
`is_inverse`	bool	prime en coin	chargeur	défaut `true`
`contract_size`	numeric(18,8)	taille	chargeur	nullable
`quote_currency`	text	devise de cotation	chargeur	nullable
`valid_from` / `valid_to`	timestamptz	période de validité de la version	chargeur	`valid_to IS NULL` = version courante
`recorded_at`	timestamptz	quand la ligne a été écrite	défaut	ne pas confondre avec `valid_from`

Trois garde-fous, et il faut savoir lequel fait quoi :

```
instrument_courant        UNIQUE (provider, instrument_name) WHERE valid_to IS NULL
                          → au plus une version courante par instrument

instrument_sans_chevauchement  EXCLUDE USING gist (instrument_name WITH =,
                                                   tstzrange(valid_from, valid_to) WITH &&)
                          → deux versions ne peuvent pas se recouvrir dans le temps
                          (exige btree_gist, migration 002)

instrument_option_complete     CHECK ...   → cohérence kind / expiry / strike / option_type
```

Questions

```
Q. Pourquoi une dimension SCD2 (valid_from/valid_to) plutot qu'un UPDATE ?
.....................................................................................
.....................................................................................

Q. Que se passe-t-il si Deribit change le contract_size d'un instrument deja connu ?
.....................................................................................
.....................................................................................

Q. Quelles sont les deux ecritures possibles, et laquelle est un bug ?
.....................................................................................
```

> Attendu : un UPDATE détruit l'historique et rend un rejeu non reproductible ; on ferme la ligne (`valid_to = now()`) et on en ouvre une nouvelle. La contrainte d'exclusion garantit qu'on ne peut pas se tromper silencieusement.

4.3 Table `silver.option_mark` — le fait principal

Grain : une ligne = une observation d'un instrument à un instant donné. Hypertable, chunk = 1 jour.

Colonne	Type	Unité	Sens	Piège
`observed_at`	timestamptz	UTC	horodatage **fournisseur**	axe du chunk
`instrument_sk`	bigint	—	FK vers la dimension	PK composite avec le temps
`mark_iv`	double	**pourcentage**	IV de marque	0 et 999 = sentinelles
`mark_price`	double	**unités du sous-jacent** (BTC/ETH)	prime	pas des dollars
`underlying_price`	double	**USD**	sous-jacent composite	souvent NULL (bande ATM)
`index_price`	double	**USD**	indice Deribit	souvent NULL pour la même raison
`bid_iv` / `ask_iv`	double	pourcentage	fourchette d'IV	renseignées seulement dans la bande ATM
`bid_price` / `ask_price`	double	unités du sous-jacent	fourchette de prime	idem
`open_interest`	double	contrats	OI	idem
`delta` … `rho`	double	—	grecques	idem
`source`	text	—	`markprice` / `ticker` / `markprice+ticker`	dit d'où vient la ligne

Index et politique :

```
PRIMARY KEY (instrument_sk, observed_at)     → unicite, et rejeu idempotent (ON CONFLICT DO NOTHING)
option_mark_instantane (observed_at DESC, instrument_sk)
                                             → la requete reelle : "toute la chaine a un instant t"
compression au-dela de 7 jours, segmentby instrument_sk, orderby observed_at DESC
                                             → ~10x, les blocs restent par instrument
pas de retention                             → l'etat echantillonne est petit, on le garde
```

Questions

```
Q. Pourquoi la cle primaire commence-t-elle par instrument_sk et non par observed_at ?
.....
.....

Q. Deux canaux alimentent mark_iv dans deux unites differentes. Comment la base l'empeche-t-elle ?
.....
.....

Q. Pourquoi bid_iv est-elle NULL pour la plupart des lignes, et pourquoi ce n'est pas un bug ?
.....
```

> Attendu : la PK donne l'unicité par instrument, l'index donne le parcours par temps. Les unités sont verrouillées par les CHECK de la migration 005 (1 à 500, plus 0 et 999 tolérés). Les NULL disent « l'instrument n'était pas dans la bande ATM à ce moment-là » — c'est une information, pas un défaut.

4.4 Table `silver.perp_tick` — le portage

Grain : une ligne = un état du perpétuel à un instant donné. Hypertable, chunk = 1 jour.

Colonne	Unité	Sens	Piège
`index_price`	USD	indice du sous-jacent	c'est le **spot de référence** des vues
`mark_price`	USD	marque du perp	`mark − index` = la base
`best_bid_price` / `best_ask_price`	USD	carnet	—
`current_funding`	**fraction par 8 h**	0.0000453 = 0,45 bp	à annualiser ×3×365
`funding_8h`	idem	taux de la période	doublon partiel de `current_funding`
`interest_rate`	fraction	composante taux	—
`open_interest`	contrats	OI	—

```
Q. Ou trouve-t-on le "spot" dans cette base, et pourquoi pas dans option_mark ?
.....
.....

Q. Que vaut un funding de 0,01 % par 8 h en annuel, et pourquoi ce n'est pas le taux sans risque ?
.....
```

> 0,01 % × 3 × 365 ≈ 10,95 %/an. Ce n'est pas le taux sans risque : c'est un prix d'équilibre entre longs et shorts du perp, qui peut être négatif. Il mesure la **base**, pas le coût de l'argent.

4.5 Table `silver.vol_index` — DVOL

Grain : une ligne = une valeur de DVOL pour un sous-jacent à un instant donné. Clé primaire `(underlying, observed_at)`, hypertable, pas de dimension : DVOL n'est pas négociable.

```
Q. A quoi sert DVOL dans le projet, et a quoi elle ne sert PAS ?
.....
.....
```

> Utile : contrôle de vraisemblance du niveau ATM, et détection de régime. Inutile : ce n'est pas une entrée du modèle, et ce n'est pas un forward.

4.6 Les quatre vues `gold`

Vue	Grain	Lit	Réponse à quelle question
`v_spot`	1 ligne par sous-jacent	`perp_tick`, `instrument`	le marché, maintenant
`v_surface`	1 ligne par instrument coté	`option_mark`, `instrument`, `v_spot`	la surface instantanée
`v_atm_terme`	1 ligne par (sous-jacent, échéance)	`v_surface`	la structure par terme ATM
`v_perp_funding`	1 ligne par tick	`perp_tick`, `instrument`	la base et le funding annualisé

Trois pièges de lecture, à connaître par cœur :

```
1. option_mark est un flux INCREMENTAL : une tranche de 5 min ne contient que ce qui a bouge.
   D'ou DISTINCT ON (instrument_sk) ... ORDER BY instrument_sk, observed_at DESC
   -> "la surface" se lit par instrument (derniere valeur), JAMAIS par tranche.

2. k_logm = ln(K / reference), ou reference = sous_jacent Deribit si publie, sinon l'indice du
   perpetuel (colonne reference_du_k). Ce n'est PAS ln(K/F) : aucun forward n'est implique ici.
   Consequence : un biais de skew croissant avec T.

3. t_annees : echeance a 08:00 UTC, base 365,25 jours. Toute autre convention decale le fit
   d'une fraction de jour -- invisible sur les courtes maturites, visible sur les longues.
```

Questions

```
Q. Pourquoi "la surface maintenant" ne peut-elle pas se lire avec un WHERE observed_at > now() - 5 min
?
.....
.....
.....

Q. Le coût de v_surface sur 1,4 M de lignes : que fait DISTINCT ON, et que faudrait-il pour aller plus
vite ?
.....
.....
```

> Réponse 2 : c'est un tri complet par instrument puis un premier par groupe. Plus vite : une table `instrument` avec un pointeur `last_mark_sk`, ou une vue matérialisée rafraîchie par le chargeur.

4.7 La table des unités (à recopier de mémoire)

Grandeur	Unité en base	Conversion	Erreur classique
IV de marque	**pourcentage** (35.61)	÷100 pour le modèle	traiter 35.61 comme 35 % de vol
prime	**unités du sous-jacent** (BTC)	× index pour des USD	la lire comme des dollars
strike, index, spot	**USD**	—	—
maturité	**années**, base 365,25, expires 08:00 UTC	—	base 365, ou 16:00
funding	**fraction par 8 h**	×3×365 pour l'annuel	le lire en pourcentage
DVOL	**pourcentage**	÷100	—
nombre de quotes	lignes	—	compter les échéances

```
Une unite fausse sur cette liste, et le fit est faux sans qu'aucune erreur ne s'affiche.
Lequel de ces pieges as-tu deja rencontre, et comment l'as-tu vu ?
.....
.....
```

4.8 Ce que la base ne contient pas (et qui bloque la calibration)

Manque	Conséquence	Où le corriger
forward par échéance	k est construit sur le spot → biais de skew en T	migration futures + `v_forward`
futures datés en `instrument`	impossible aujourd'hui : le CHECK impose `expiry IS NULL` pour `kind <> 'option'`	migration 009
tables de calibration	supprimées en 004 ; aucun fit n'est stocké	schéma v2 (voir ci-dessous)
table de snapshot/surface	chaque fit repartirait de `v_surface`, non rejouable	idem
index « surface à un instant t »	`v_surface` scanne 1,4 M lignes	vue matérialisée ou pointeur

Le schéma de calibration **v2** à écrire (proposition, à valider) :

```
calibration_run
  run_id, snapshot_id, engine_git_sha, started_at, finished_at, status,
  rho, eta, gamma, nu, rmse_total, min_g, n_quotes_used, conventions   <- linear|inverse

calibration_slice
  run_id, expiry, t_years, theta, forward, n_quotes, rmse, min_g

quote_fit
  run_id, expiry, strike, is_call, k_logm, iv_market, iv_model, residual, weight

surface_snapshot                      <- ce qui manque le plus
  snapshot_id, observed_at, underlying, k_grid, filter_json, forward_source,
  n_instruments, sha256
```

```
Ce que je changerais a cette proposition, et pourquoi :
.....................................................................................
.....................................................................................
```

4.9 Le point a verifier avant le sprint S1

Le CHECK `instrument_option_complete` interdit aujourd'hui de stocker un **future daté** :

```
kind <> 'option'  =>  expiry IS NULL AND strike IS NULL AND option_type IS NULL
```

Or un future BTC-27MAR26 a une échéance et pas de strike. La migration 009 devra donc relâcher la contrainte en distinguant trois cas : option (expiry + strike + type), future (expiry, sans strike), perpetual (ni l'un ni l'autre).

```
Reecris la contrainte correcte :
CHECK (
.....................................................................................
.....................................................................................
)
```

4.10 Dix requêtes à savoir écrire de tête

```
1. Le spot de reference              : select * from gold.v_spot;
2. La derniere marque d'un instrument: select * from silver.option_mark
                                       where instrument_sk = $1 order by observed_at desc limit 1;
3. La surface du jour               : select * from gold.v_surface where expiry > current_date;
4. La structure par terme ATM        : select * from gold.v_atm_terme order by expiry;
5. La base du perp                  : select * from gold.v_perp_funding order by observed_at desc
limit 5;
6. Le nombre d'echeances par devise  : select underlying, count(distinct expiry) from gold.v_surface
group by 1;
7. La fraicheur du silver            : select max(observed_at) from silver.option_mark;
8. Les vols aberrants                : select * from silver.option_mark where mark_iv > 200 limit 20;
9. Le volume par jour                : select time_bucket('1 day', observed_at) d, count(*) from
                                       silver.option_mark group by 1 order by 1 desc;
10. La taille des chunks             : select chunk_name, range_start, range_end
                                       from timescaledb_information.chunks
                                       where hypertable_name = 'option_mark' order by range_start desc
limit 5;
```

```
Laquelle de ces dix requetes ne sais-tu pas ecrire sans aide ?
.....................................................................................
```

4.11 Expliquer la base en 60 secondes

À répéter à voix haute, chronométré. Si tu dépasses 60 secondes, c'est que tu expliques le code plutôt que la structure.

```
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
```

> Squelette attendu : trois schémas ; bronze = fichiers rejouables ; silver = une dimension SCD2 (instrument) et trois tables de faits en hypertables, grains et unités ; gold = quatre vues de lecture ; `observed_at` est l'heure du fournisseur, donc un rejeu est idempotent ; la table `option_mark` porte l'IV en pourcentage et la prime en coin — les deux pièges d'unités ; et il manque le forward et les tables de calibration.

---

5. Les décisions ouvertes

Une fiche par décision. Tant que la colonne « mesure » est vide, ce n'est pas une décision : c'est une préférence. Les fiches sont volontairement laides — c'est un formulaire, pas une présentation.

D1 — Convention de prix : linéaire USD ou inverse coin

Contexte. Deribit publie une `mark_iv` calculée dans *son* modèle. Le moteur actuel price en dollars avec un forward. Si les deux conventions diffèrent, on ajuste une surface qui ne reproduit pas les prix de la place.

Options

#	Option	Conséquence	Effort
A	Black-76 linéaire, conversion parité	simple, réutilise tout le code existant	0
B	Black inverse (coin)	reproduit les prix Deribit si c'est bien leur convention	1 kernel
C	Put linéaire sur X = 1/S, strike 1/K	exact, réutilise la machinerie linéaire	1 kernel

La mesure qui tranche. Reconstruire `mark_price` depuis `mark_iv` avec A, B et C sur un échantillon (≥ 5 000 quotes, ≥ 10 échéances, ailes incluses) et comparer au prix publié.

```
Écart médian A (points de vol) : ..............   max : ..............
Écart médian B (points de vol) : ..............   max : ..............
Écart médian C (points de vol) : ..............   max : ..............
```

Décision — date : ..... choix : .....

```
Pourquoi, en une phrase mesurable :
.....................................................................................
```

D2 — Coordonnée de Dupire

Contexte. La volatilité locale dérive de la surface en variance totale. La coordonnée doit être celle dans laquelle l'IV ajustée est définie, sinon on reproduit une autre surface que celle qu'on a ajustée.

Critère. Écart entre le pricer retenu (D1) et le moteur linéaire converti, sur les ailes.

```
Seuil acceptable : ..............   Mesuré : ..............
```

Décision — date : ..... choix : .....

```
.....................................................................................
```

D3 — Structure par terme de θ

Contexte. Aujourd'hui `θ(T) = ν·T`. Sur SPX, cela pinçait `η/η_max` à 0,9999 et l'erreur ATM croissait avec T. Trois voies :

#	Option	Ce que ça donne	Risque
A	garder `θ = νT`	1 paramètre, propre, mais rigide	fit structurellement biaisé
B	`θ_i` libre par maturité, contraint monotone	colle la structure par terme réelle	1 paramètre par slice
C	`θ` paramétrique (puissance + plateau)	compromis, 2-3 paramètres	paramètres non identifiables

```
Ce que j'observe sur BTC (erreur ATM par echeance) :
.....................................................................................
Choisi : ..........   parce que : ..................................................
```

D4 — Source du forward crypto

#	Option	Avantage	Défaut
A	indice du perpétuel	disponible en continu	ce n'est pas un forward : biais = base
B	futures datés + interpolation	le vrai forward par échéance	il faut collecter une nouvelle famille d'instruments
C	parité call-put par échéance	auto-cohérent avec les quotes	bruit si les deux côtés sont illiquides

```
Mesure : ecart entre A, B et C sur 3 echeances (en bp) :
.....................................................................................
Decision : ..........   date : ............
```

D5 — Le trou de données du 3 au 10 novembre

Contexte. Le bronze est intact, le silver s'arrête au 3 novembre 07:30 UTC (disque plein). Le chargeur ne regarde que les 3 dernières heures.

#	Option	Conséquence
A	ignorer le trou, reprendre au présent	le plus simple ; l'historique silver a un trou documenté
B	rejouer tout le bronze en `--since-hours 200`	~8 jours de fichiers, plusieurs heures de calcul
C	rejouer un échantillon (1 fichier sur 12)	reconstruit la couverture, pas le détail

```
Decide : ..........   parce que : ..........................................................
Si A : ou est-ce ecrit (README, note, commentaire de migration) ? ..........................
```

D6 — Capacité et rétention

Faits mesurés. Base 5,8 Go. Bronze 25 Go sur l'hôte. Disque Colima 59 Go (21 Go utilisés après ménage). Hôte : 31 Go libres sur 926.

```
Ce qui grossit le plus vite : ..................................................................
Regle de retention que j'adopte : ..............................................................
Alerte a declencher au-dela de : .......... % du disque
```

---

6. Fiches fonction — à réécrire à la main

Pour chaque fonction : écris le contrat **avant** de regarder le code, puis compare. La colonne « ce que je changerais » est la plus importante : c'est là que naissent les lots suivants.

F1 — `impliedVol`

```
signature : ...................................................................................
entrees : .....................................................................................
sorties : .....................................................................................
erreurs possibles : ...........................................................................

pseudo-code de memoire :
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................
.....................................................................................

trois cas limites que je teste : ..............................................................
.....................................................................................
ce que je changerais : ........................................................................
```

F2 — `calibrateSSVI`

```
signature : ..........................................................................
parametres libres et bornes : ..........................................................
fonction objectif : ....................................................................
algorithme : ...........................................................................

pseudo-code de memoire :
.......................................................................................
.......................................................................................
.......................................................................................
.......................................................................................

pourquoi une reparametrisation bornee : ................................................
ce que je changerais : .................................................................
```

F3 — `forwardDiscrete`

```
signature : ..........................................................................
formule : ..............................................................................
modele de dividende utilise : ..........................................................

ce qui manque pour des dividendes discrets reels :
.......................................................................................
.......................................................................................
ce que je changerais : .................................................................
```

F4 — `pde::price` (Crank-Nicolson)

```
signature : ..........................................................................
grille : ..............................................................................
conditions aux bords : .................................................................
traitement du temps (theta-schema) : ...................................................
style americain : ......................................................................

ce qui donne une erreur d'ordre 1 sans qu'on le voie :
.......................................................................................
ce que je changerais : .................................................................
```

F5 — `MarketForwardCurve::F` (S0)

```
signature : ..........................................................................
interpolation : ........................................................................
extrapolation : ........................................................................
invariants : ...........................................................................

ce qui se passe si deux maturites sont egales :
.......................................................................................
ce que je changerais : .................................................................
```

F6 — `load_marks.main`

```
signature : ..........................................................................
fenetre de lecture : ...................................................................
transaction : ..........................................................................
idempotence : ..........................................................................

ce qui se passe si un fichier est corrompu au milieu du lot :
.......................................................................................
ce que je changerais : .................................................................
```

---

7. Journal de session

Une ligne par séance. Ce qui compte n'est pas « j'ai lu X » mais **ce que je saurais réexpliquer**.

Date	Ce que j'ai lu / fait	Ce que je sais maintenant expliquer	Décision prise	À vérifier

Annexe A — Runbook de l'incident du 10 novembre 2026

Symptôme. Le conteneur `db` redémarre en boucle : `FATAL: could not write lock file "postmaster.pid": No space left on device`.

Diagnostic, dans cet ordre.

```
1. df -h /var/lib/docker (dans le VM Colima)      -> 59 Go, 0 disponible
2. docker system df                               -> volumes 35,8 Go, build cache 9,7 Go
3. du -sh <volume pgdata>/_data/base              -> 34 Go
4. psql -c "select datname, pg_size_pretty(pg_database_size(datname)) from pg_database"
   -> neuf bases load_test_* de ~5,6 Go chacune, laissees par les tests d'integration
```

Ce qui a marché, dans l'ordre.

```
docker builder prune -af                  # 9,7 Go
psql -f drop_test_dbs.sql                 # 34 Go  (les bases load_test_*)
docker compose restart db
```

Ce qu'il ne faut pas faire. `docker compose down -v` : cela supprime le volume `pgdata`, donc la base. Et ne jamais supprimer le volume pour « faire de la place ».

Les trois actions de prévention.

1. Les tests qui créent `load_test_*` doivent **supprimer la base** en fin de test (teardown), et non seulement fermer la connexion.
2. `ops/mini-stack-guard.sh` doit alerter si le disque du VM dépasse 85 %.
3. Un balayage hebdomadaire des bases `load_test_*` de plus de 24 h.

La leçon. Un collecteur « healthy » ne dit rien de la chaîne. Le seul indicateur de santé qui compte est `select max(observed_at) from silver.option_mark` comparé à `now()`. À mettre dans l'alerte, avant même l'état des conteneurs.

Annexe B — Commandes utiles

```
# tunnel depuis le MacBook
ssh -N -L 5432:127.0.0.1:5432 macmini-ai@macmini-ai.local

# psql sur le mini
docker compose exec db psql -U vse -d volsurface

# fraicheur des donnees
select max(observed_at) from silver.option_mark;

# build et tests C++
cmake -S . -B build && cmake --build build -j4 && ctest --test-dir build --output-on-failure

# tests Python
./.venv/bin/pytest -q tests/python

# demonstration synthetique (aucune donnee requise)
python3 tools/make_synthetic_market.py --out data/csv/SYN
./build/calibrate_spx_minimal data/csv/SYN

# rendu de ce cahier
scripts/build_cours_pdf.py docs/cahier_complet.md docs/cahier_complet.pdf
```

Annexe C — Où est quoi

Ce que je cherche	Fichier
les courbes de taux et dividendes	`include/data/RateCurve.hpp`, `DivCurve.hpp`
le forward (modèle et marché)	`include/data/ForwardCurve.hpp`
le noyau de prix	`include/pricing/PricingKernel.hpp`
le cœur SSVI	`include/calibration/SSVI.hpp`, `src/calibration/SSVI.cpp`
l'inversion d'IV	`src/calibration/ImpliedVol.cpp`
volatilité locale (Dupire)	`src/calibration/LocalVol.cpp`
PDE Crank-Nicolson	`src/pde/PDEPricer.cpp`, `Tridiag.cpp`
le schéma de base	`sql/001` … `sql/008` (ordre d'application)
le chargeur bronze → silver	`python/jobs/load_marks.py`
le collecteur WebSocket	`python/collect/deribit_ws.py`
les vues de lecture	`sql/008_couches.sql`
le cours complet	`docs/cours_complet.pdf`
le marché synthétique	`tools/make_synthetic_market.py`

Annexe D — Glossaire minimal

Terme	Sens dans ce projet
variance totale	`w(k,T) = σ²(k,T)·T`, la grandeur que SSVI paramètre
log-moneyness	`k = ln(K/F)`
calendaire	le non-arbitrage entre deux maturités (w croissante en T)
papillon	le non-arbitrage dans une maturité (densité ≥ 0)
`g(k)`	dénominateur de Dupire ; c'est aussi le critère de Durrleman
`η_max`	borne supérieure de η telle que la condition suffisante tienne
inverse (option)	payoff réglé dans la devise du sous-jacent (BTC), pas en dollars
SCD2	dimension historisée par période de validité
hypertable	table partitionnée par le temps (TimescaleDB)
bronze / silver / gold	brut rejouable / conformé / vues de lecture

Annexe E — Checklist de fin de phase 1

```
[ ] le moteur lit un snapshot produit par le pipeline, pas un CSV herite
[ ] k est construit sur le forward de l'echeance, et la source du forward est tracee
[ ] la convention de prix (D1) est mesuree puis documentee, avec le chiffre
[ ] un fit est rejoue a l'identique depuis un snapshot_id
[ ] les parametres et les residus sont en base, avec le sha du binaire
[ ] la fraicheur des donnees est surveillee, et l'alerte part sur max(observed_at)
[ ] le disque est surveille, et les bases de test se suppriment toutes seules
[ ] le cahier est a jour : chaque decision a un numero, une date et une mesure
```